S

Adding a Product in SmartPOS

How the "New product" feature works from the screen to the database: the app design, the database design, every validation rule, and the logic in between, with flowcharts.

1 · App design | 2 · Flow | 3 · Validation | 4 · Logic | 5 · Database design

---

SmartPOS · Angular 22 web app · ASP.NET Core 10 API · EF Core · PostgreSQL 16
Every screen, response and SQL statement in this document was captured from the running application on 4 November 2026.

Contents

1. **At a glance**
   What happens when you press "Create product", in one picture
2. **App design**
   The screens, the form fields, how the form behaves, and the front-end files
3. **End-to-end flow**
   The request travelling through every layer, as a sequence diagram
4. **Validation**
   Four layers of checks, a rule-by-rule table, and the real error responses
5. **Logic and functionality**
   What the server does step by step, as a flowchart, with the code
6. **Database design**
   The three tables, their columns, keys, constraints and the SQL that runs
7. **Responses and errors reference**
8. **Things to know and possible improvements**
9. **Files involved**

0 At a glance

A shop owner fills in the **New product** form and presses **Create product**. The browser sends **one** request, `POST /api/products`. The server checks it, then writes **three rows** in **one transaction**: the product, its stock record, and a stock-history entry for the opening stock.

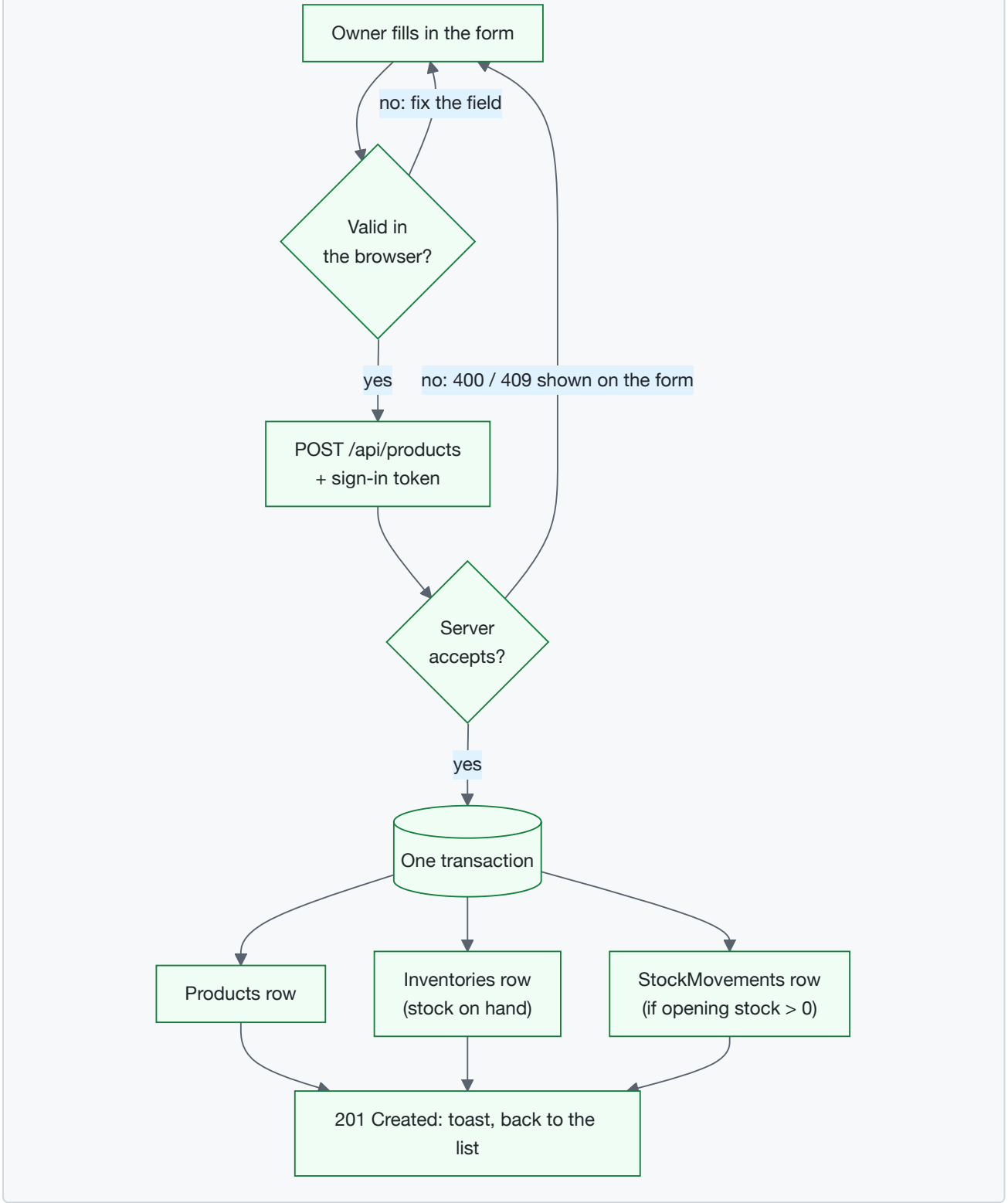


Figure 0.1. The whole feature in one picture

KEY FACTS

- The product always belongs to the **shop of the signed-in user**. The shop comes from the sign-in token, never from the form.
- Checks happen at **four levels**: the browser, the API's field rules, the business rules (no duplicate SKU or barcode in the same shop), and the database's own constraints.
- Stock is kept in a **separate table** from the product, and every stock change is recorded in a **ledger**. That's why one product creates up to three rows.
- If any part fails, **nothing** is saved: the three inserts succeed or fail together.

1 App design

1.1 Where the feature lives

The products page (`/products`) lists the catalogue and has a green **New product** button. It opens the form at `/products/new`. The same form component also edits a product at `/products/:id/edit`. Both pages are inside the signed-in part of the app, so a visitor who isn't signed in is sent to the login page first.

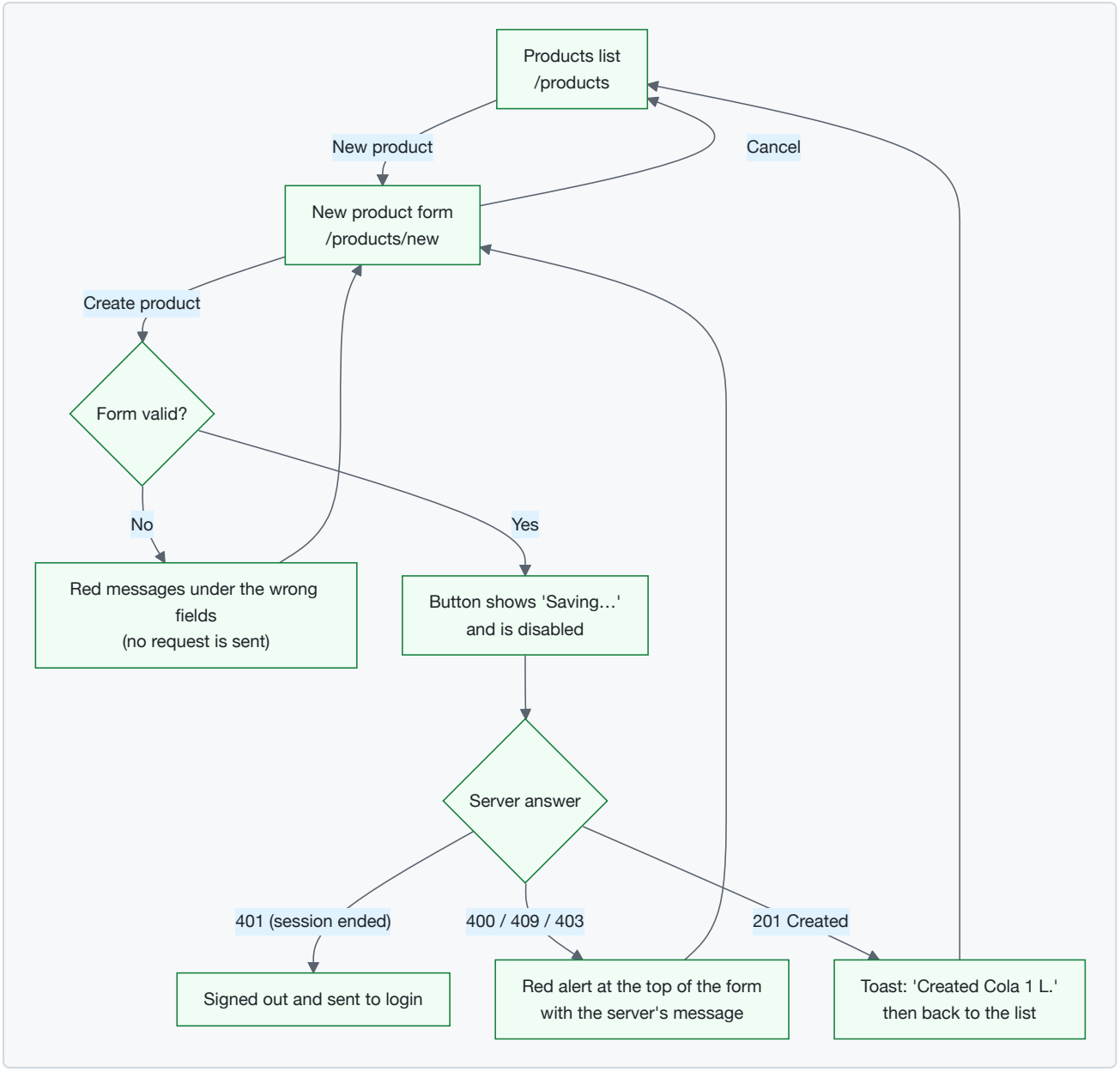


Figure 1.1. Screen flow

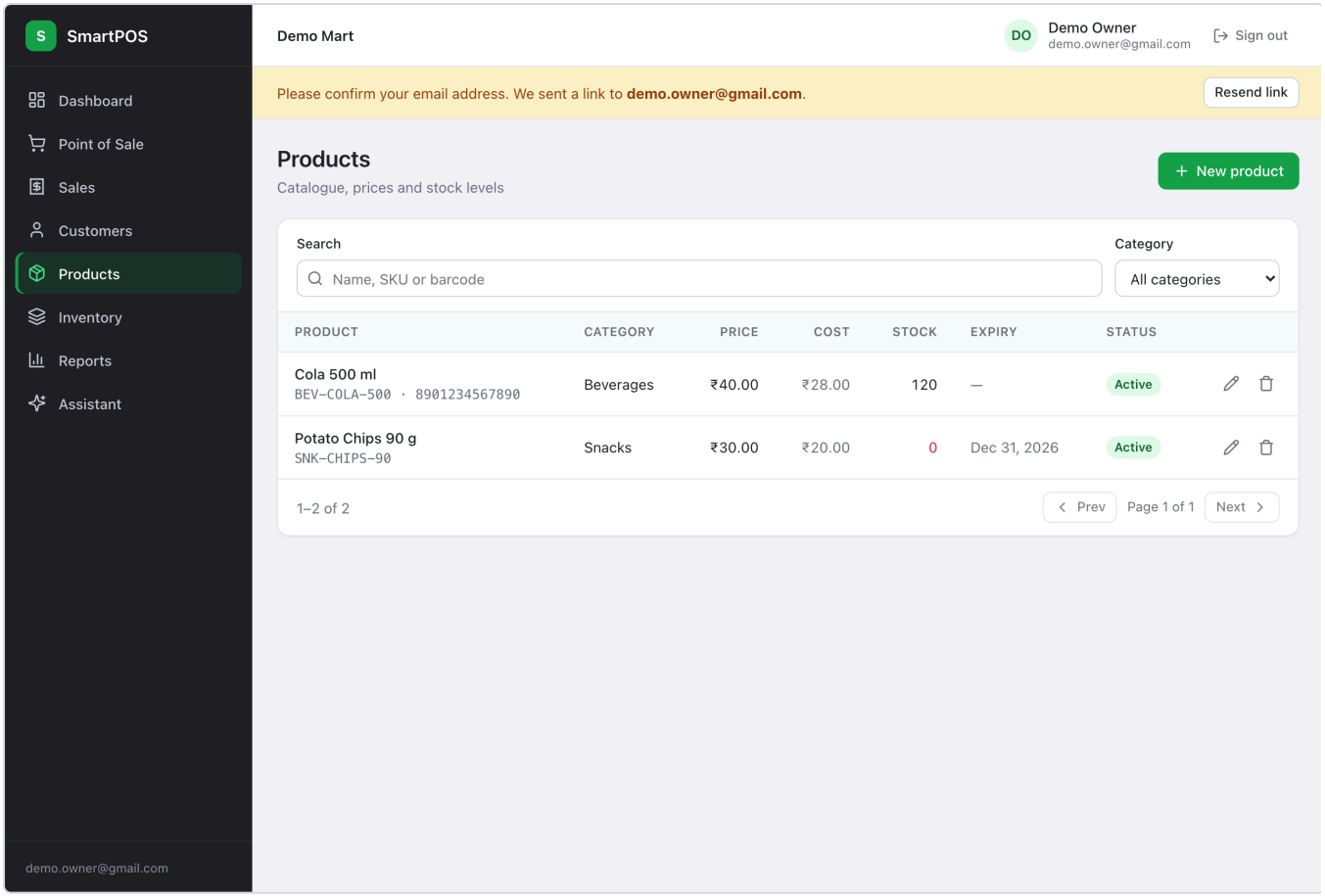


Figure 1.2. The products page with the New product button (top right)

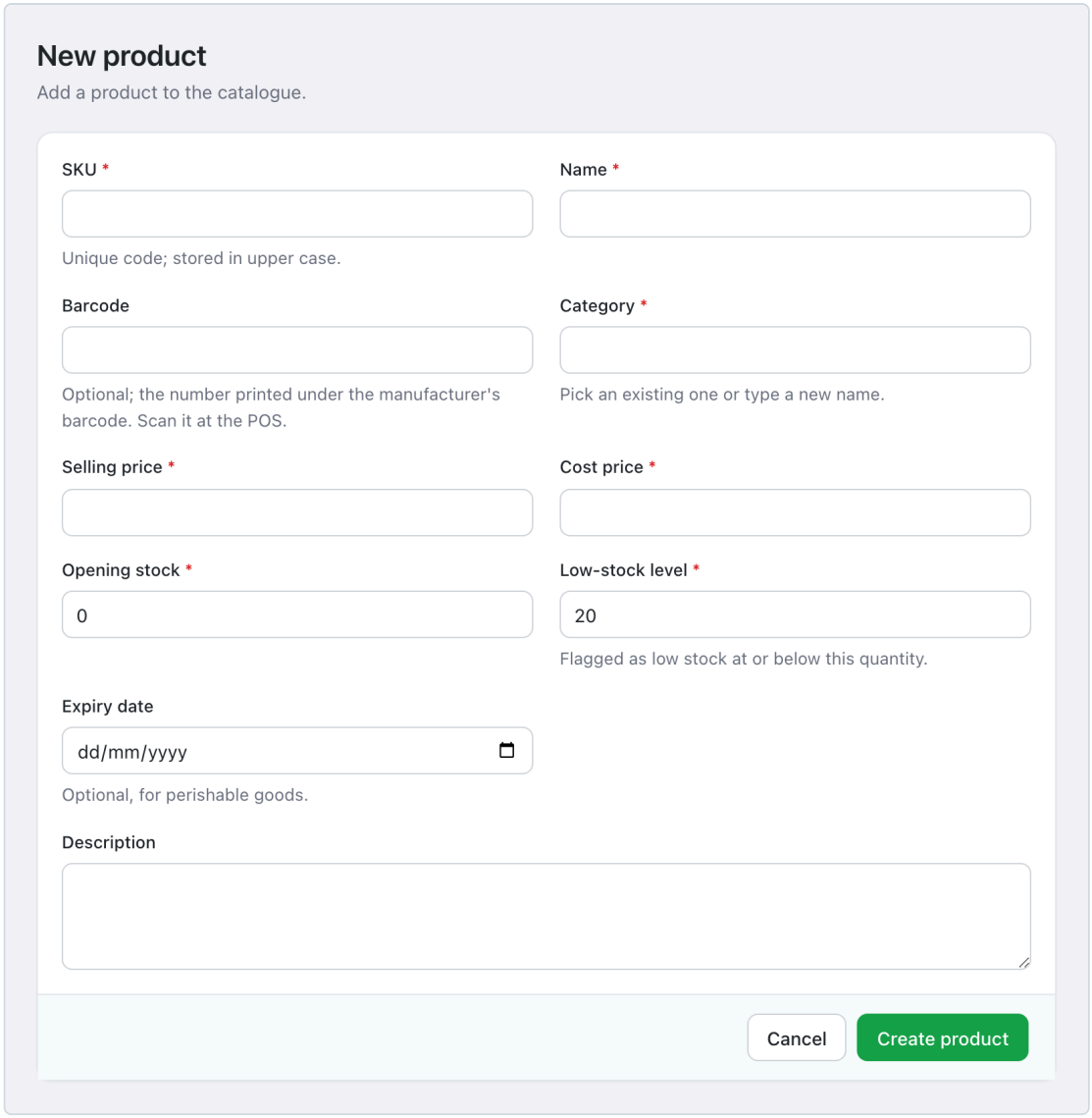


Figure 1.3. The empty New product form. Opening stock starts at 0 and the low-stock level at 20.

1.2 The form fields

Field	Required	Default	What it is	Help text on screen
SKU	Yes	empty	The shop's own product code, e.g. `BEV-COLA-500`	"Unique code; stored in upper case."
Name	Yes	empty	What appears in the POS and on bills	
Barcode	No	empty	The manufacturer's number under the barcode, so it can be scanned at the POS	"Optional; the number printed under the manufacturer's barcode."
Category	Yes	empty	Free text. The box suggests existing categories; typing a new name creates a new category.	"Pick an existing one or type a new name."
Selling price	Yes	empty	Price charged at the till	
Cost price	Yes	empty	What the shop paid, used for profit reports	
Opening stock	Yes	0	Units on hand right now. *Only on the create form.*	
Low-stock level	Yes	20	At or below this many units the product is flagged as low	"Flagged as low stock at or below this quantity."

Field	Required	Default	What it is	Help text on screen
Expiry date	No	empty	For perishable goods	"Optional, for perishable goods."
Description	No	empty	Free notes	

1.3 How the form behaves

- **Errors appear only when relevant:** a field turns red once it has been touched, or when you press Create with something wrong. A fresh form isn't covered in red.
- **Pressing Create with mistakes sends nothing.** All fields are marked as touched, so every problem shows at once (Figure 1.4). The test run confirmed that **0 requests** were sent with an invalid form.
- **Tidying before sending:** text is trimmed, and empty optional fields (barcode, description, expiry) are sent as `null` rather than `""`.
- **Category suggestions** come from `GET /api/categories` when the form opens. That's the list of category names already used in this shop, with product counts.
- **While saving**, the button reads "Saving…" and is disabled, so a double-click can't create two products.
- **Errors from the server** (for example a duplicate SKU) appear as a red alert at the top of the form, and the typed values stay in place (Figure 1.5).
- **On success**, a toast says "Created <name>." and the app returns to the list, where the new product appears (Figure 1.6).

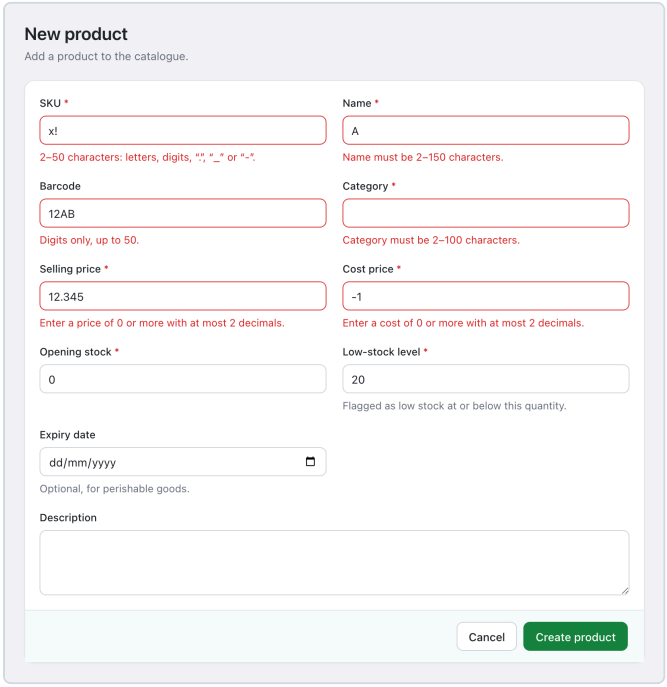


Figure 1.4. Browser checks: every wrong field explains itself. Nothing was sent to the server.

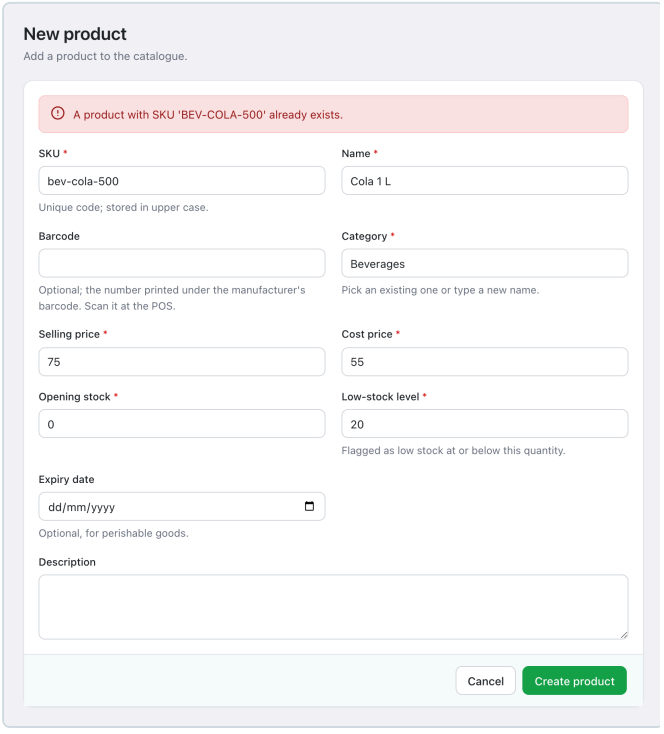


Figure 1.5. The fields passed the browser checks, but the server found SKU `BEV-COLA-500` already in this shop (409).

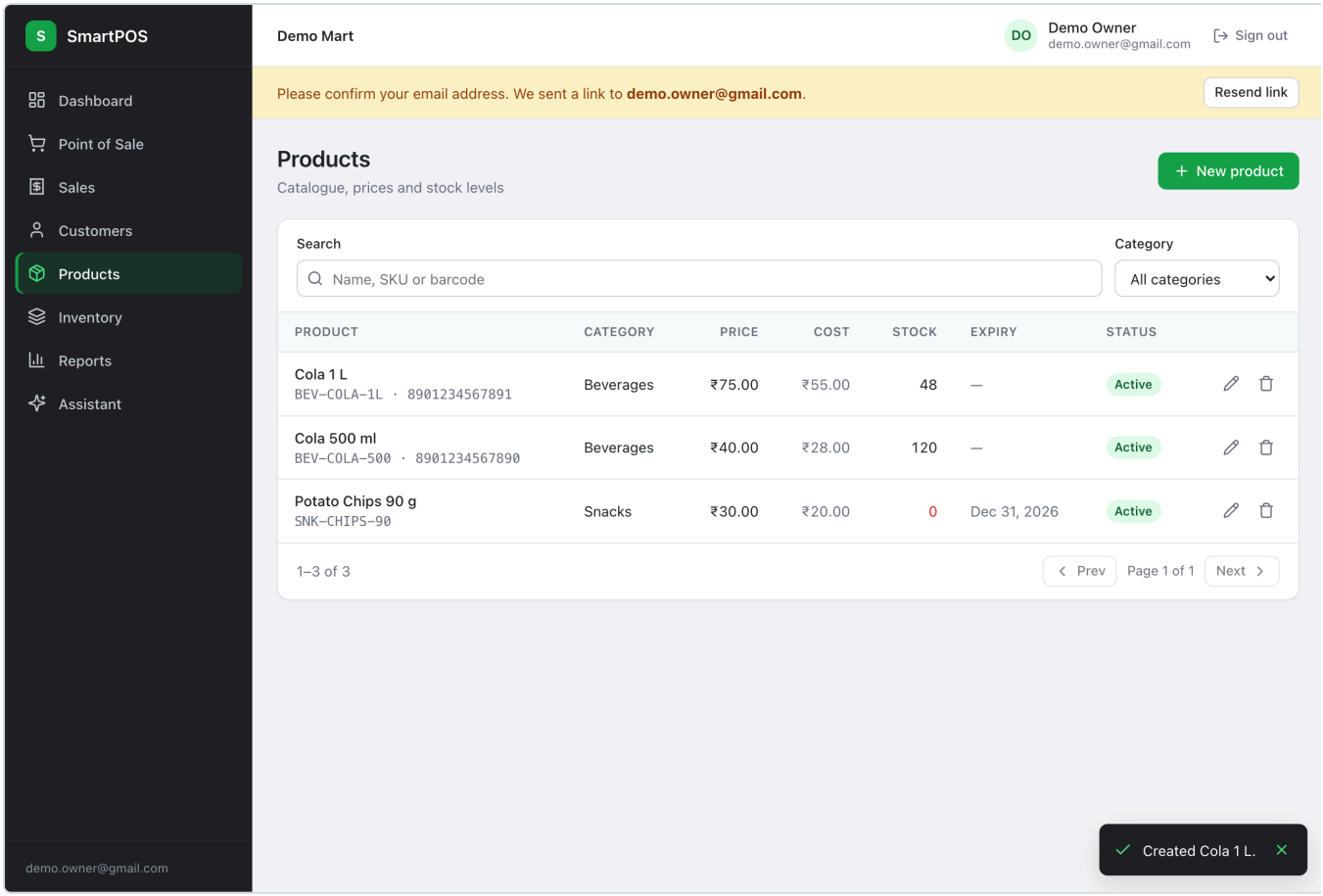


Figure 1.6. Success: the toast at the bottom right, and "Cola 1 L" in the list with 48 in stock. The SKU typed as `BEV-COLA-1L` is shown as stored.

1.4 Create vs edit

	New product (`/products/new`)	**Edit product (`/products/:id/edit`)**
Request	`POST /api/products`	`PUT /api/products/{id}`
Stock field	**Opening stock**, editable	**Stock on hand**, read-only: "Change stock from Inventory so every change is recorded."
Active checkbox	Not shown; new products are always active	Shown: "Active (available in the POS)"
Button	Create product	Save changes

1.5 Front-end design

The page is an Angular **standalone component** using a **reactive form**: every field and its rules are declared in code, in one place. State such as "loading", "saving" and "save error" is held in **signals**, so the screen updates exactly where something changed.

```
// product-form.ts: the form and its browser-side rules
protected readonly form = this.fb.group({
  sku:  ['', [Validators.required, Validators.minLength(2), Validators.maxLength(50),
Validators.pattern(/^[A-Za-z0-9][A-Za-z0-9._-]*$/)]],
  name: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(150)]],
  barcode: ['', [Validators.maxLength(50), Validators.pattern(/^\d*$/)]],
  description: ['', [Validators.maxLength(1000)]],
  category: ['', [Validators.required, Validators.minLength(2), Validators.maxLength(100)]],
  price:     this.fb.control<number | null>(null, [Validators.required, Validators.min(0),
Validators.max(9999999.99), money]),
  costPrice: this.fb.control<number | null>(null, [Validators.required, Validators.min(0),
Validators.max(9999999.99), money]),
  initialStock:      this.fb.control<number | null>(0,  [Validators.required, Validators.min(0),
Validators.max(1_000_000), Validators.pattern(/^\d+$/)]),
  lowStockThreshold: this.fb.control<number | null>(20, [Validators.required, Validators.min(0),
Validators.max(1_000_000), Validators.pattern(/^\d+$/)]),
  expiryDate: [''],
  isActive: [true],
});

/** money: at most 2 decimal places (mirrors the API's [MaxDecimalPlaces(2)]). */
function money(control) { const v = control.value; return v === null || Math.abs(v * 100 - Math.round(v *
100)) < 1e-6 ? null : { money: true }; }

// save(): stop if invalid, tidy the values, send, then toast + navigate or show the error
protected save(): void {
  if (this.form.invalid) { this.form.markAllAsTouched(); return; }      // nothing is sent
  const v = this.form.getRawValue();
  const input: ProductInput = {
    sku: v.sku.trim(), name: v.name.trim(), barcode: v.barcode.trim() || null,
    description: v.description.trim() || null, category: v.category.trim(),
    price: v.price!, costPrice: v.costPrice!, lowStockThreshold: v.lowStockThreshold!,
    expiryDate: v.expiryDate || null, initialStock: v.initialStock!,
  };
  this.saving.set(true);
  this.products.create(input).subscribe({
    next: (p) => { this.toast.success(`Created ${p.name}.`); void this.router.navigate(['/products']); },
    error: (err) => { this.saveError.set(errorMessage(err, 'Could not save the product.'));
this.saving.set(false); },
  });
}
```

How the request leaves the browser. `ProductsService.create()` calls `HttpClient.post('/api/products', input)`. On the way out, the **auth interceptor** adds the header `Authorization: Bearer <token>`. If the server ever answers 401, the interceptor signs the user out. `errorMessage()` turns any error response into one sentence for the red alert.

2 End-to-end flow

Here is the same click followed through every layer of the system. The back end is built in four layers, each in its own project. The arrows point inwards: inner layers know nothing about outer ones.

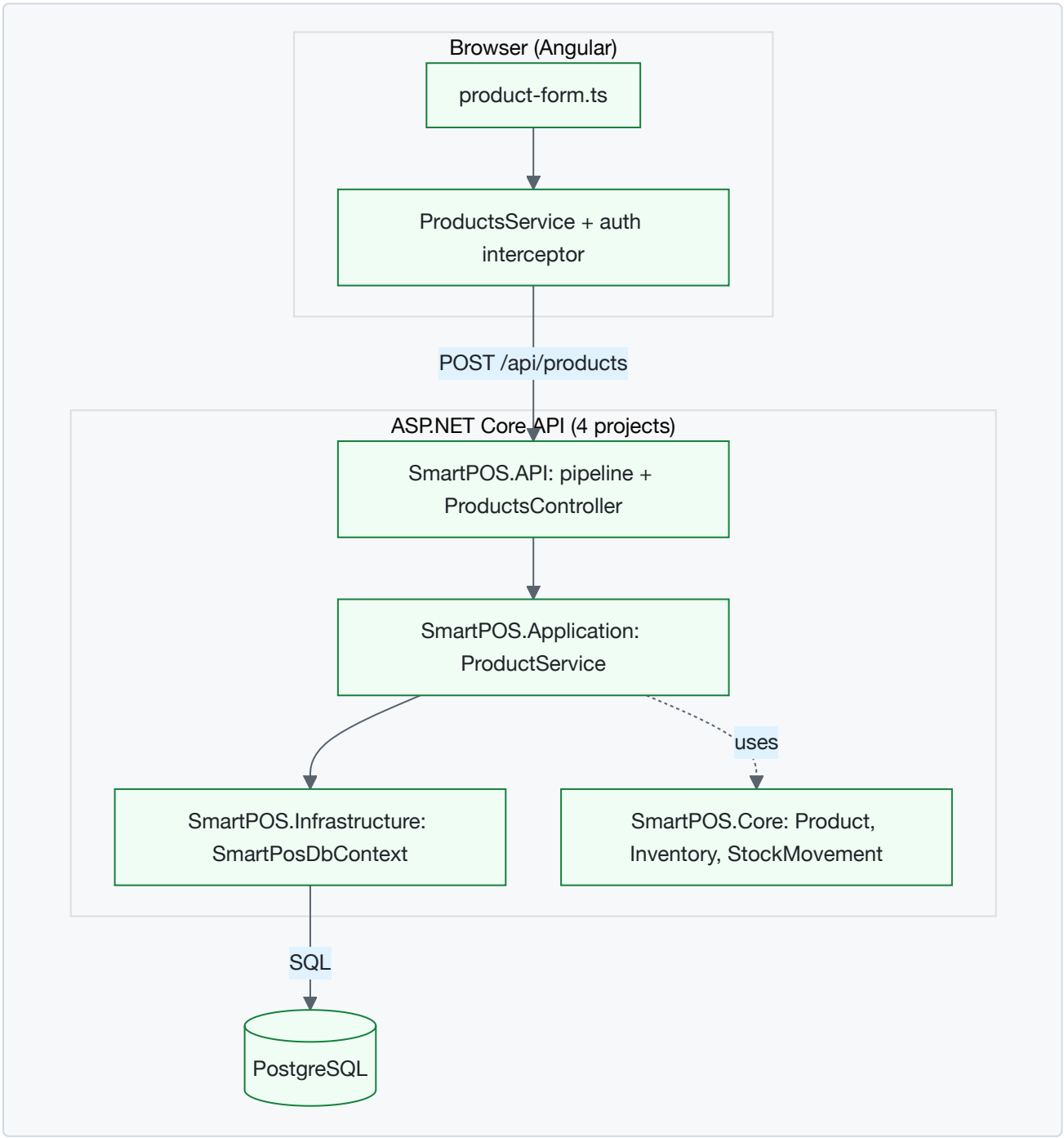


Figure 2.1. The layers involved

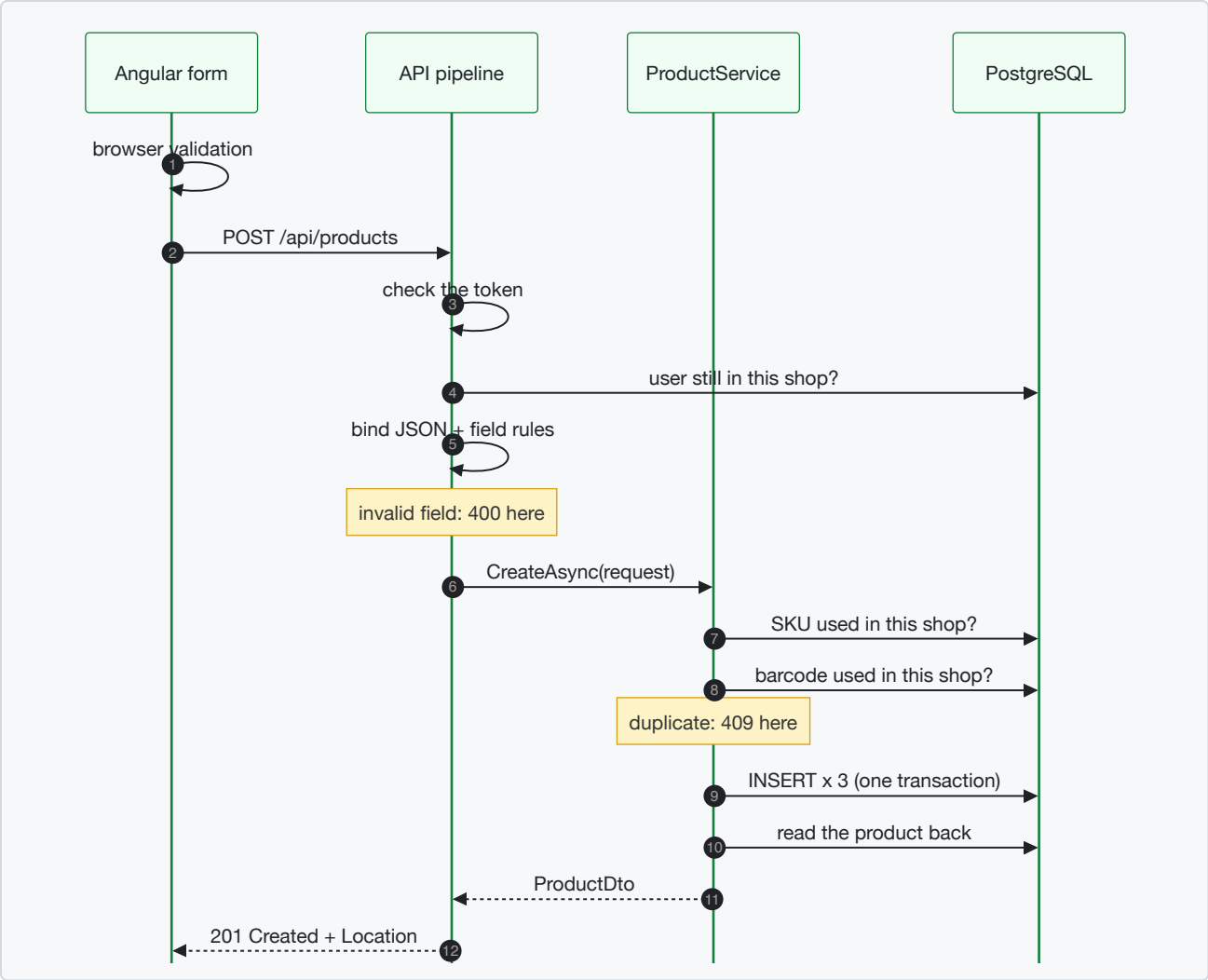


Figure 2.2. One "Create product" click, step by step

What each step is for

#	Step	Where	Why
3–4	Check the token, then check the user still exists in that shop	`ApiServiceCollectionExtensions.cs` (JWT setup, `OnTokenValidated`)	Only signed-in owners can add products; a deleted account's old token stops working
5	Turn the JSON into a `CreateProductRequest` and check its attributes	`ProductDtos.cs` + `[ApiController]`	Bad shapes are refused before any business code runs
6	Hand over to the service	`ProductsController.Create`	Controllers stay thin; the rules live in one place
7–8	Duplicate checks, limited to this shop	`ProductService.EnsureSkuIsFree / EnsureBarcodeIsFree`	A clear message instead of a database error
9	Build the rows and save once	`ProductService.CreateAsync`	All three rows commit together, or none do

#	Step	Where	Why
10–12	Re-read and return	`ProductService.GetAsync`	The response shows exactly what was stored (upper-cased SKU, trimmed name…)

3 Validation

Every value is checked more than once, at different layers, each with a different job. The browser gives instant, friendly feedback. The server **never trusts the browser**: someone could call the API directly, so it repeats every rule. The database is the last line of defence, and it holds even if application code has a bug.

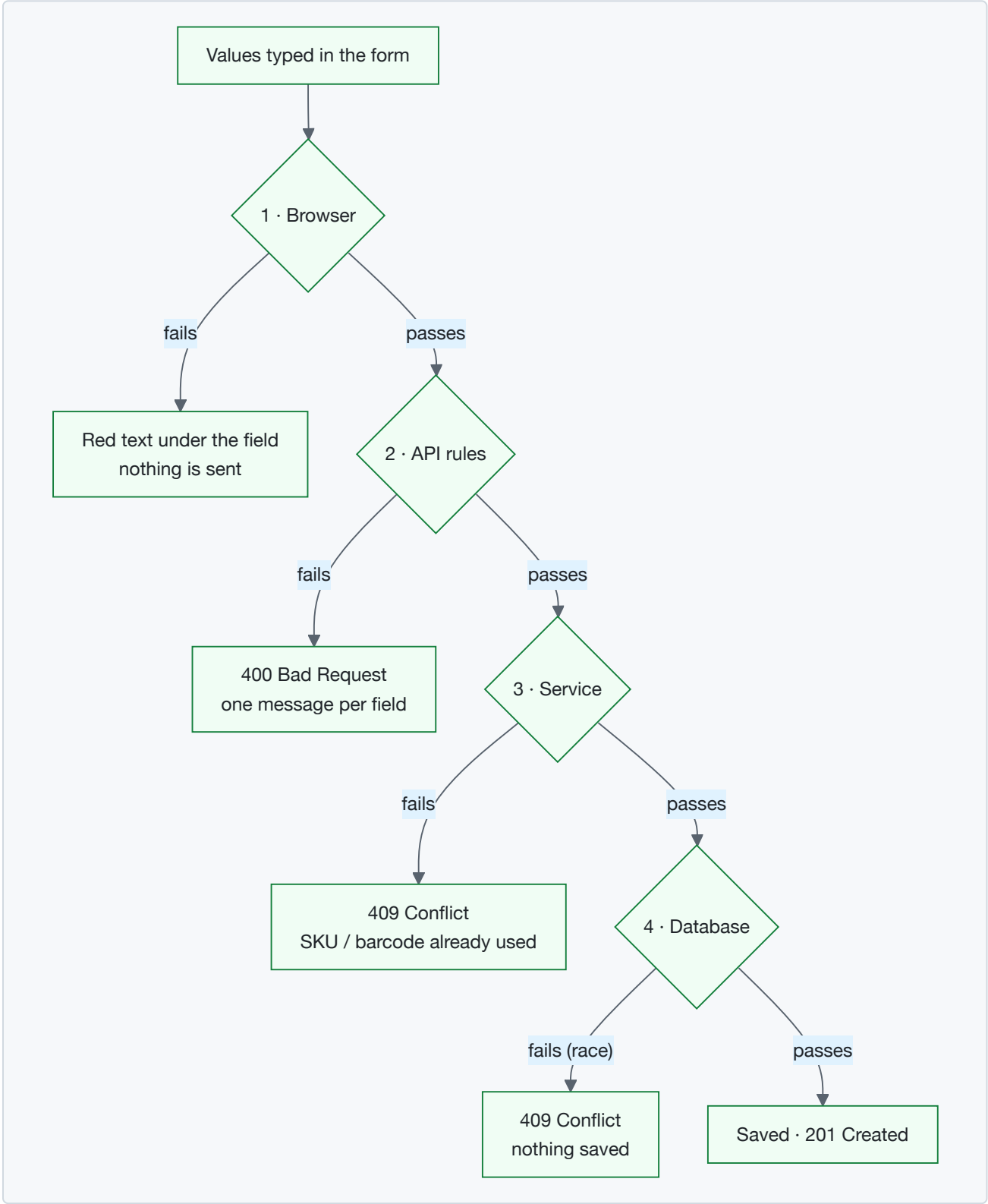


Figure 3.1. The four validation layers

3.1 Every rule, field by field

Field	1 · Browser	2 · API field rules	3 · Service	4 · Database
SKU	required; 2–50 chars; starts with a letter or digit, then letters, digits, `.` `_` `-`	same: `[Required, StringLength(50, Min 2)]`, `[RegularExpression]`	trimmed and **upper-cased**; must not exist in this shop	`varchar(50) NOT NULL`; unique `(ShopId, Sku)`
Name	required; 2–150	`[Required, StringLength(150, Min 2)]`	trimmed	`varchar(150) NOT NULL`
Barcode	optional; digits only; ≤ 50	`[StringLength(50)]`, `[RegularExpression(^\d*$)]`	blank → `null`; if given, must not exist in this shop	`varchar(50)` nullable; unique `(ShopId, Barcode)`, and many NULLs are allowed
Category	required; 2–100	`[Required, StringLength(100, Min 2)]`	trimmed	`varchar(100) NOT NULL`
Description	≤ 1000	`[StringLength(1000)]`	blank → `null`, trimmed	`varchar(1000)` nullable
Selling price	required; 0 – 9,999,999.99; ≤ 2 decimals	`[Range(0, 9999999.99)]`, `[MaxDecimalPlaces(2)]`		`numeric(18,2)`; `CHECK (Price >= 0)`
Cost price	same as price	same as price		`numeric(18,2)`; `CHECK (CostPrice >= 0)`
Opening stock	required; whole number 0 – 1,000,000	`[Range(0, 1000000)]`	> 0 also writes a ledger row	`Inventories.StockQuantity`; `CHECK (>= 0)`
Low-stock level	required; whole number 0 – 1,000,000	`[Range(0, 1000000)]`, default 20		`Inventories.LowStockThreshold`; `CHECK (>= 0)`
Expiry date	optional date	must parse as a date (`DateOnly?`)		`date` nullable
Shop	not in the form	not in the request	taken from the sign-in token	`ShopId NOT NULL`, FK → `Shops`

WHY THE SAME RULE APPEARS TWICE

Layers 1 and 2 hold the same limits on purpose. The browser copy is for speed and friendliness. The server copy is the one that counts, because a request can come from anywhere (a script, Postman, a modified page). The server copies are attributes on `CreateProductRequest`; ASP.NET checks them automatically before the controller method runs.

3.2 Real responses captured from the API

Several fields wrong at once → 400. Sent: SKU `"x"`, empty name, barcode `"12AB"`, category `"B"`, price `12.345`, cost `-1`, opening stock `-5`. Every problem is reported in one go, and **no SQL ran**:

```
HTTP/1.1 400 Bad Request
{ "title": "One or more validation errors occurred.", "status": 400,
  "errors": {
    "Sku":          [ "The field Sku must be a string with a minimum length of 2 and a maximum length of
50." ],
    "Name":         [ "The Name field is required.", "The field Name must be a string with a minimum length
of 2 and a maximum length of 150." ],
    "Price":        [ "Price must have at most 2 decimal places." ],
    "Barcode":      [ "Barcode may contain only digits." ],
    "Category":     [ "The field Category must be a string with a minimum length of 2 and a maximum length
of 100." ],
    "CostPrice":    [ "The field CostPrice must be between 0 and 9999999.99." ],
    "InitialStock": [ "The field InitialStock must be between 0 and 1000000." ] } }
```

Situation (tested)	Status	Message
SKU `Bev-Cola-500` when `BEV-COLA-500` exists in this shop (the check ignores case)	409	A product with SKU 'BEV-COLA-500' already exists.
New SKU, but barcode `8901234567890` is already used in this shop	409	A product with barcode '8901234567890' already exists.
SKU sent with spaces around it, `" bev-cola-500 "`, straight to the API	400	SKU may contain only letters, digits, '.', '_' and '-'. *(The web form trims first, so users never see this.)*
No sign-in token	401	Unauthorized
A **different shop** creates the same SKU and barcode	201	Allowed: codes are unique *per shop*
The other shop asks for this shop's product by ID	404	Product 1 was not found. (It can't even tell the product exists.)

4 Logic and functionality

Once the request passes the field rules, `ProductService.CreateAsync` does the work. Here it is as a flowchart, then as code.

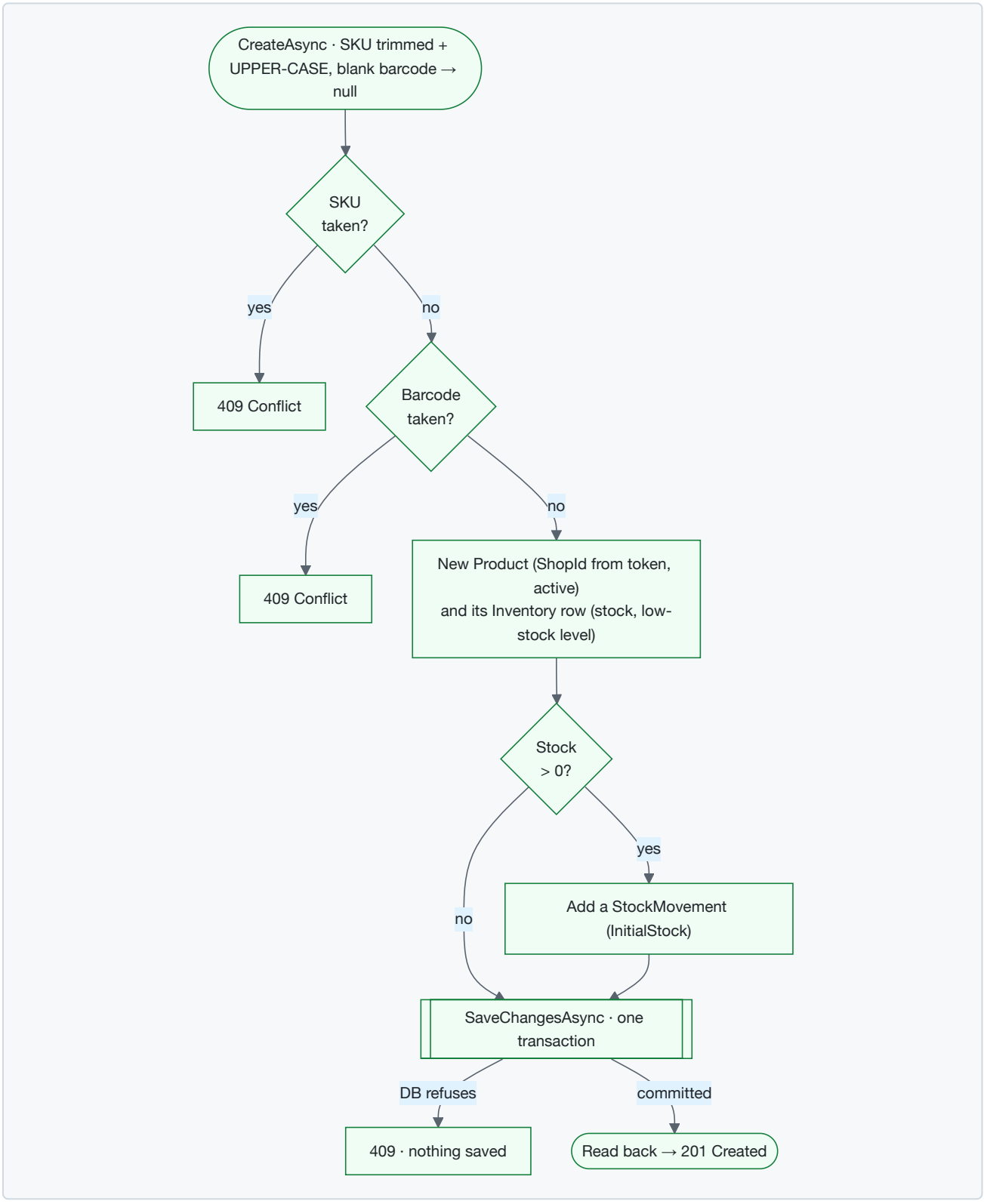


Figure 4.1. ProductService.CreateAsync

```
// SmartPOS.Application/Products/ProductService.cs
public async Task<ProductDto> CreateAsync(CreateProductRequest request, CancellationToken cancellationToken)
{
    var sku = NormalizeSku(request.Sku);                        // "bev-cola-500" → "BEV-COLA-500"
    var barcode = Clean(request.Barcode);                       // "" or "  " → null
    await EnsureSkuIsFree(sku, excludeProductId: null, cancellationToken);        // 409 if taken
    await EnsureBarcodeIsFree(barcode, excludeProductId: null, cancellationToken); // 409 if taken

    var now = calendar.UtcNow;
    var product = new Product
    {
        ShopId = currentUser.ShopId,                                // ← from the token, never from the
request
        Sku = sku, Barcode = barcode,
        Name = request.Name.Trim(), Description = Clean(request.Description), Category =
request.Category.Trim(),
        Price = request.Price, CostPrice = request.CostPrice, ExpiryDate = request.ExpiryDate,
        IsActive = true, CreatedAt = now, UpdatedAt = now,
        Inventory = new Inventory                                   // stock lives in its own table
        {
            ShopId = currentUser.ShopId, StockQuantity = request.InitialStock,
            LowStockThreshold = request.LowStockThreshold, LastUpdated = now
        }
    };
    db.Products.Add(product);

    if (request.InitialStock > 0)                                  // the ledger starts with the opening stock
        db.StockMovements.Add(new StockMovement
        {
            ShopId = currentUser.ShopId, Product = product,
            QuantityChange = request.InitialStock, QuantityAfter = request.InitialStock,
            Type = StockMovementType.InitialStock, Reason = "Opening stock",
            UserId = currentUser.UserId, CreatedAt = now
        });

    // One SaveChanges = one implicit transaction: product, inventory row and movement go in together.
    await db.SaveChangesAsync(cancellationToken);
    return await GetAsync(product.Id, cancellationToken);
}

private async Task EnsureSkuIsFree(string sku, int? excludeProductId, CancellationToken ct)
{
    // Friendly pre-check; the unique index on (ShopId, Sku) is the real guarantee.
    if (await db.Products.AnyAsync(p => p.Sku == sku && p.Id != excludeProductId, ct))
        throw new ConflictException($"A product with SKU '{sku}' already exists.");
}
```

4.1 What gets stored: a worked example

This is exactly what the form sent in the test run, and what came back.

Field	Sent by the browser	Stored and returned	Why it changed
sku	`"bev-cola-500"`	`"BEV-COLA-500"`	upper-cased, so *bev-cola-500* and *BEV-COLA-500* count as the same code
name	`" Cola 500 ml "`	`"Cola 500 ml"`	trimmed
description	`""`	`null`	empty means "none"
barcode	`"8901234567890"`	`"8901234567890"`	
price / costPrice	`40.00` / `28.00`	`40.00` / `28.00`	
initialStock	`120`	`stockQuantity: 120`	stored in the Inventories table

Field	Sent by the browser	Stored and returned	Why it changed
lowStockThreshold	`24`	`24`, `isLowStock: false`	120 > 24, so not low
(not sent)		`id: 1`, `isActive: true`, `createdAt` = `updatedAt`	set by the server

4.2 The rules behind the logic

- **Shop isolation.** Every product query automatically gets `WHERE "ShopId" = <your shop>` (an EF Core *global query filter*). So the duplicate checks only look at *your* products. Two shops may both sell `BEV-COLA-500`, and one shop can never read another's product.
- **Stock can't be typed in later.** Opening stock is set once, here. After that, stock changes only through sales and inventory adjustments, and each one writes a ledger row. The edit form shows stock as read-only, and the update request has no stock field at all.
- **All or nothing.** The product, its inventory row and its first movement are saved by a single `SaveChangesAsync`, which EF Core wraps in one transaction. If the database refuses any of them, none are kept.
- **The answer is read back from the database** (`GetAsync`), so the screen shows what was really stored.
- **201 Created with a Location header.** The controller returns `CreatedAtAction(nameof(Get), new { id }, product)`, which gives `Location: http://…/api/products/1`.

5 Database design

Adding a product touches three tables. They hang off `Shops`: every row carries a `ShopId`, which is how many shops share one database safely.

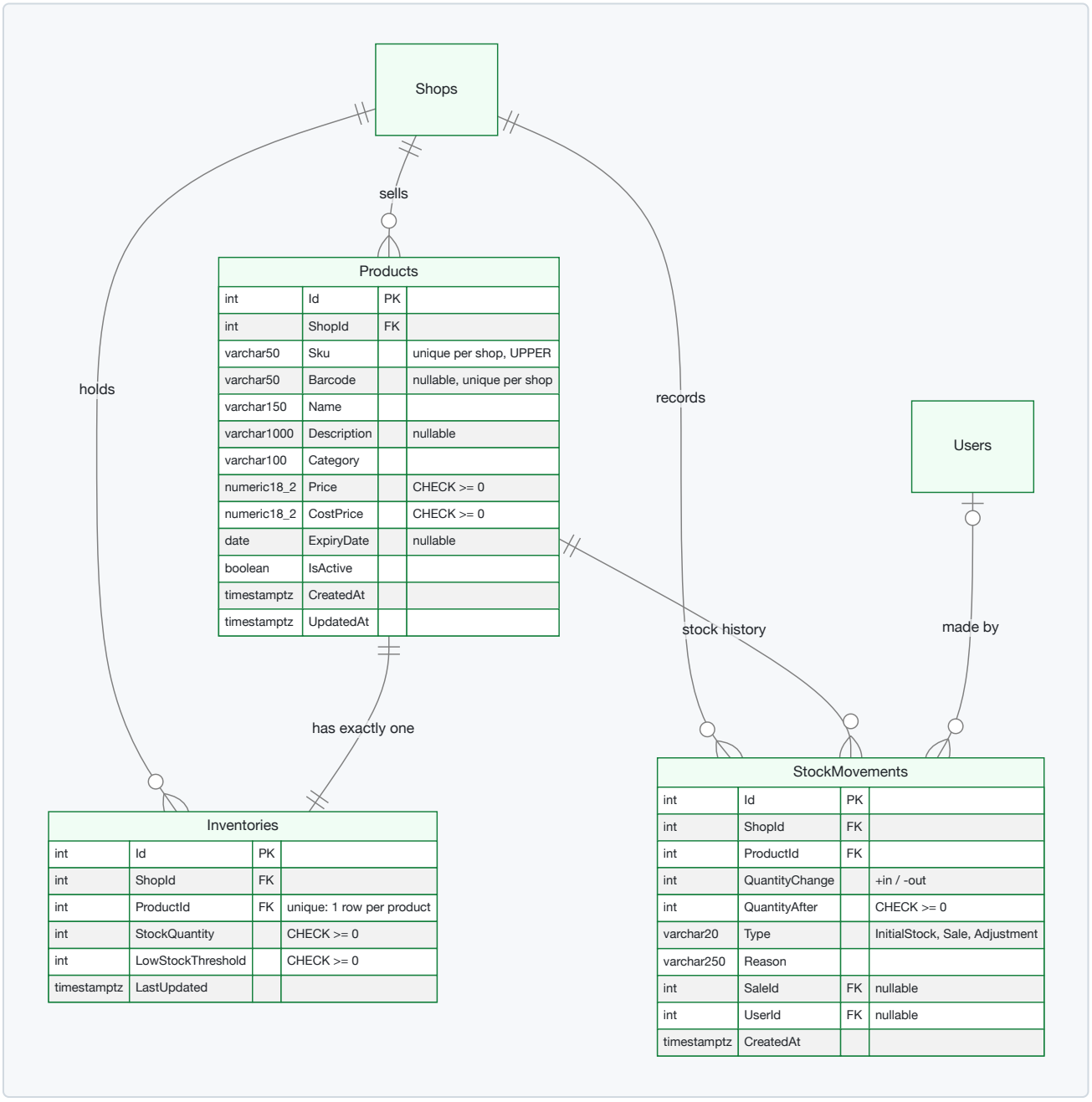


Figure 5.1. The tables involved and how they relate

5.1 Why three tables?

Table	Holds	Design reason
Products	What the item *is*: code, name, prices, category, expiry	Editing a product (rename, new price) can never change stock by accident
Inventories	How many are on hand *now*, and the low-stock level	A small row that sales update often, with a database rule that stock can't go below 0

Table	Holds	Design reason
StockMovements	Every change ever: +120 opening, –2 sale, +50 delivery…	An append-only ledger: you can always explain the current stock, and the movements add up to it

Category is plain text on the product, not a separate table. A new category exists as soon as a product uses it, and `GET /api/categories` simply lists the distinct names (`GROUP BY "Category"`).

5.2 Keys, indexes and constraints (as created in PostgreSQL)

Name	Definition	What it guarantees
`IX_Products_ShopId_Sku`	UNIQUE (`ShopId`, `Sku`)	No two products with the same SKU in one shop, even if two requests race
`IX_Products_ShopId_Barcode`	UNIQUE (`ShopId`, `Barcode`)	Same for barcodes; products without one (NULL) don't collide
`IX_Products_ShopId_Category`	(`ShopId`, `Category`)	Fast category filter on the products page
`IX_Products_Name`	(`Name`)	Fast sorting by name
`CK_Products_Price_NonNegative`, `CK_Products_CostPrice_NonNegative`	CHECK (`Price` ≥ 0), CHECK (`CostPrice` ≥ 0)	No negative prices, whatever the code does
`IX_Inventories_ProductId`	UNIQUE (`ProductId`)	Exactly one stock row per product
`CK_Inventories_StockQuantity_NonNegative`	CHECK (`StockQuantity` ≥ 0)	Stock can never be negative
`CK_Inventories_LowStockThreshold_NonNegative`	CHECK (`LowStockThreshold` ≥ 0)	
`CK_StockMovements_QuantityAfter_NonNegative`	CHECK (`QuantityAfter` ≥ 0)	The ledger never records negative stock
`FK_Products_Shops_ShopId`	→ `Shops(Id)` ON DELETE RESTRICT	A shop with products can't be deleted
`FK_Inventories_Products_ProductId`	→ `Products(Id)` ON DELETE CASCADE	Deleting a product removes its stock row
`FK_StockMovements_Products_ProductId`	→ `Products(Id)` ON DELETE CASCADE	…and its history (only possible for never-sold products)
`FK_StockMovements_Users_UserId`	→ `Users(Id)` ON DELETE RESTRICT	Records who added the stock

5.3 The SQL that actually runs

Captured from the API's database log while creating "Cola 500 ml". Values are sent as parameters (`@p0`…), never pasted into the SQL, which is what makes SQL injection impossible here. Note the automatic `"ShopId" = @ef_filter__CurrentShopId` in the checks.

```
-- 1. duplicate checks (only in this shop)
SELECT EXISTS (SELECT 1 FROM "Products" AS p WHERE p."ShopId" = @ef_filter__CurrentShopId AND p."Sku" =
@sku)
SELECT EXISTS (SELECT 1 FROM "Products" AS p WHERE p."ShopId" = @ef_filter__CurrentShopId AND p."Barcode" =
@barcode)

-- 2. SaveChangesAsync: one transaction, two round trips
INSERT INTO "Products" ("Barcode", "Category", "CostPrice", "CreatedAt", "Description", "ExpiryDate",
"IsActive", "Name", "Price", "ShopId", "Sku", "UpdatedAt")
VALUES (@p0, @p1, @p2, @p3, @p4, @p5, @p6, @p7, @p8, @p9, @p10, @p11) RETURNING "Id";

INSERT INTO "Inventories" ("LastUpdated", "LowStockThreshold", "ProductId", "ShopId", "StockQuantity")
VALUES (@p12, @p13, @p14, @p15, @p16) RETURNING "Id";
INSERT INTO "StockMovements" ("CreatedAt", "ProductId", "QuantityAfter", "QuantityChange", "Reason",
"SaleId", "ShopId", "Type", "UserId")
VALUES (@p17, @p18, @p19, @p20, @p21, @p22, @p23, @p24, @p25) RETURNING "Id";

-- 3. read it back (stock joined in from Inventories)
SELECT p."Id", p."Sku", p."Barcode", p."Name", …, i0."StockQuantity", i0."LowStockThreshold", …
FROM "Products" AS p LEFT JOIN (SELECT … FROM "Inventories" AS i WHERE i."ShopId" =
@ef_filter__CurrentShopId) AS i0 ON p."Id" = i0."ProductId"
WHERE p."ShopId" = @ef_filter__CurrentShopId AND p."Id" = @id LIMIT 2
```

5.4 The rows written in the test run

Products

Id	ShopId	Sku	Name	Barcode	Price / Cost	ExpiryDate
1	1	BEV-COLA-500	Cola 500 ml	8901234567890	40.00 / 28.00	NULL
2	1	SNK-CHIPS-90	Potato Chips 90 g	NULL	30.00 / 20.00	2026-12-31
3	**2**	BEV-COLA-500	Cola 500 ml	8901234567890	40.00 / 28.00	NULL

Inventories

ProductId	ShopId	StockQuantity	LowStockThreshold
1	1	120	24
2	1	0	10
3	2	120	24

StockMovements

ProductId	Change	After	Type	Reason
1	+120	120	InitialStock	Opening stock
3	+120	120	InitialStock	Opening stock

Product 2 was created with opening stock 0, so it has an inventory row but **no** movement. Product 3 belongs to the second shop and reuses the same SKU and barcode, which the per-shop unique indexes allow.

6 Responses and errors reference

Status	When	Produced by	What the user sees
201 Created	Product saved	`ProductsController.Create` → `CreatedAtAction`	Toast "Created <name>." and back to the list
400 Bad Request	A field breaks a rule (length, pattern, range, decimals)	`[ApiController]` automatic model validation	Normally caught by the browser first; otherwise the messages in the red alert
401 Unauthorized	No token, expired token, or the user no longer exists	JWT authentication + fallback policy	Signed out and sent to the login page
409 Conflict	SKU or barcode already used in this shop	`ProductService` → `ConflictException`	"A product with SKU '…' already exists."
409 Conflict	The database's unique index catches a race the pre-check missed	`DbUpdateException` → `GlobalExceptionHandler`	"The change conflicts with existing data. Reload and try again."
0 (no answer)	The API isn't running	the browser	"Cannot reach the SmartPOS server. Check that the API is running."
500	An unexpected bug	`GlobalExceptionHandler`	"The server hit an unexpected error." (details only in the server log)

Every error body uses the standard **problem details** format (`title`, `status`, `detail` or `errors`), and the web app's `errorMessage()` reads it the same way everywhere.

7 Things to know and possible improvements

These were found while testing for this document. None of them is reachable through the web form unless noted, and none was changed.

A MISSING PRICE BECOMES ₹0.00 (API ONLY)

The API's price fields have a range but no `[Required]`, so a request that leaves out `price` and `costPrice` is accepted and stored as 0.00. Tested: `{"sku":"MIN-ONLY","name":"Minimal item","category":"Test"}` → **201** with price 0.00, cost 0.00, stock 0, low-stock level 20. The web form requires both prices, so users can't hit this. *Fix:* make the two properties `decimal?` with `[Required]`.

AN EXPIRY DATE IN THE PAST IS ACCEPTED

Tested: expiry `2020-01-01` → **201**. This affects the web form too. It may be intended (recording stock that has already expired), and the dashboard and AI assistant do report expired products. *Option:* warn in the form when the date is before today.

SPACES AROUND THE SKU

The API checks the SKU pattern *before* trimming, so `" bev-cola-500 "` sent directly is refused (400). The web form trims first, so this only affects direct API callers. Trimming in the request model (or allowing surrounding spaces in the pattern) would make the API as forgiving as the form.

RULES LIVE IN TWO PLACES

The browser rules in `product-form.ts` copy the server rules in `ProductDtos.cs` by hand. If one changes, change the other. The server copy is the one that protects the data.

8 Files involved

Layer	File	Role in "add a product"
Web app (frontend/src/app)	`app.routes.ts`	Route `products/new` → the form, lazy-loaded inside the signed-in shell
	`features/products/product-list.html`	The **New product** button
	`features/products/product-form.ts / .html`	The form, its browser rules, tidying, saving, messages
	`core/api/products.service.ts`	`create()` → `POST /api/products`; `CategoriesService.list()` for suggestions
	`core/auth/auth.interceptor.ts`	Adds the Bearer token; signs out on 401
	`core/http/http-utils.ts`, `core/models.ts`	`errorMessage()`; the `ProductInput` and `Product` types
API	`Controllers/ProductsController.cs`	`[HttpPost] Create` → service → 201 + Location
	`Infrastructure/ApiServiceCollectionExtensions.cs`	JWT checks, "must be signed in" by default, JSON setup
	`Infrastructure/GlobalExceptionHandler.cs`	Turns `ConflictException` / `DbUpdateException` into 409
Application	`Products/ProductDtos.cs`	`CreateProductRequest` with its validation attributes; `ProductDto`
	`Products/ProductService.cs`	`CreateAsync`: normalise, duplicate checks, build rows, save, read back
	`Common/MaxDecimalPlacesAttribute.cs`	The "at most 2 decimals" rule
Core	`Entities/Product.cs`, `Inventory.cs`, `StockMovement.cs`	The three record types
Infrastructure	`Persistence/SmartPosDbContext.cs`	The shop filter on every query; saving
	`Persistence/Configurations/Product…, Inventory…, StockMovementConfiguration.cs`	Column lengths, unique indexes, CHECK constraints, foreign keys
	`Persistence/Migrations/…_InitialCreate.cs`	Creates the three tables in PostgreSQL
Tests	`IntegrationTests/ProductAndSaleTests.cs`	Duplicate SKU (any case), invalid product → 400, barcode unique per shop, shops can't see each other's products

HOW THIS DOCUMENT WAS CHECKED

Everything here was taken from the code and then confirmed against the running system on a throwaway copy of the database (not your real one): the screenshots come from the real web app; the responses, the SQL and the stored rows come from real requests; and the browser was observed sending **no** request while the form was invalid.